

АННОТАЦИЯ

В данной работе были изучены основные правила построения конечных автоматов, их разновидности, были изучены основные особенности автоматов Мили и Мура, а также расписаны основные правила построения управляющих автоматов для автоматизированных систем управления (АСУ). Были изучены правила построения таблицы переходов и графов переходов, а также минимизации конечных автоматов

В ходе практической части работы разработанный в теоретической части конечный автомат был реализован на языке программирования C++. В коде были реализованы переходы состояний, а также реализован вывод основной информации для диспетчера на экран (в консоль).

Курсовая работа состоит из 50 страниц и включает в себя: 6 таблиц и 5 рисунков.

Основные термины: конечный автомат, автомат Мили, автомат Мура, группы состояний, таблица переходов, граф переходов.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 ТЕХНОЛОГИЧЕСКАЯ ПОСТАНОВКА ЗАДАЧИ	4
2 РАЗРАБОТКА ИНФОРМАЦИОННОГО ОБЕСПЕЧЕНИЯ ЗАДАЧИ.....	11
2.1 Описание конечного автомата по состояниям. Группы состояний	11
2.2 Таблица переходов. Граф переходов.....	17
2.3 Минимизация конечного автомата	21
3 РАЗРАБОТКА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ ЗАДАЧИ.....	26
3.1. Реализация на языке C++.....	26
3.2 Реализация автомата на триггерах.....	29
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	35
Приложение	36

ВВЕДЕНИЕ

Курсовая работа является неотъемлемой частью обучения обучающихся различных специальностей. Целью курсовой работы является получение и закрепление навыков и умений, которые должны быть изучены в ходе изучения той или иной дисциплины.

Курсовая работа по дисциплине «Теория конечных автоматов» является актуальной для обучения студентов по направлению «Автоматизация технологических процессов и производств», так как отдельные части автоматизированного производства и даже само автоматизированное производство возможно описать с помощью абстрактной математической модели – конечного автомата. С помощью конечного автомата возможно упростить понимание поведения тех или иных процессов. А с помощью модификаций конечного автомата можно безопасно модифицировать производственные линии без необходимости физически расширять производство со своими рисками и последствиями.

1 ТЕХНОЛОГИЧЕСКАЯ ПОСТАНОВКА ЗАДАЧИ

Задание формируется индивидуально для каждого студента по его ФИО следующим образом.

1. Выписывается фамилия (а при необходимости, имя и отчество) студента.
2. Гласные буквы заменить на нули, согласные – на единицы.
3. Полученные выражения группируются на 4 слова по три цифры.

Пример составления задания.

Пусть ФИО студента - Цой Иван Никифорович. В этом случае ФИО студента соответствует цифровое описание:

ФИО студента	Ц	о	й	И	в	а	н	Н	и	к	и	ф
Код управления	1	0	1	0	1	0	1	1	0	1	0	1

Таким образом, получаем 4 слова по три цифры: 101 010 110 101.

Первое и третье слова обозначают номера резервуаров R_i (рис. 1), из которых в контейнер K подается материал. Второе и четвертое слова указывают на номер бункера V_j , в который подается полученная смесь из контейнера K .

Кодирование адресов осуществляется следующим образом.

Номера резервуаров R_i (первое и третье слова) кодируются позиционным способом. В автомате три резервуара R_i (R_1 , R_2 и R_3).

Количество единиц в команде определяет количество позиций резервуара, из которых материал будет поступать в контейнер K .

Расположение единиц в слове указывает на номера резервуаров. Младший разряд соответствует резервуару № 1, средний - резервуару № 2, старший - резервуару № 3.

Например, команда «101» указывает на 1 и 3 резервуары, команда «110» – на второй и третий, команда «001» указывает на 1 резервуар.

Номер бункера V_j (второе и четвертое слова) кодируется двоичным кодом.

Двоичное число, переведенное в десятичный код, указывает на номер бункера. Например, 010 соответствует бункеру *B2*, число 101- бункеру *B5*, число 100 - бункеру *B4*.

Для справки можно воспользоваться таблицей 1.

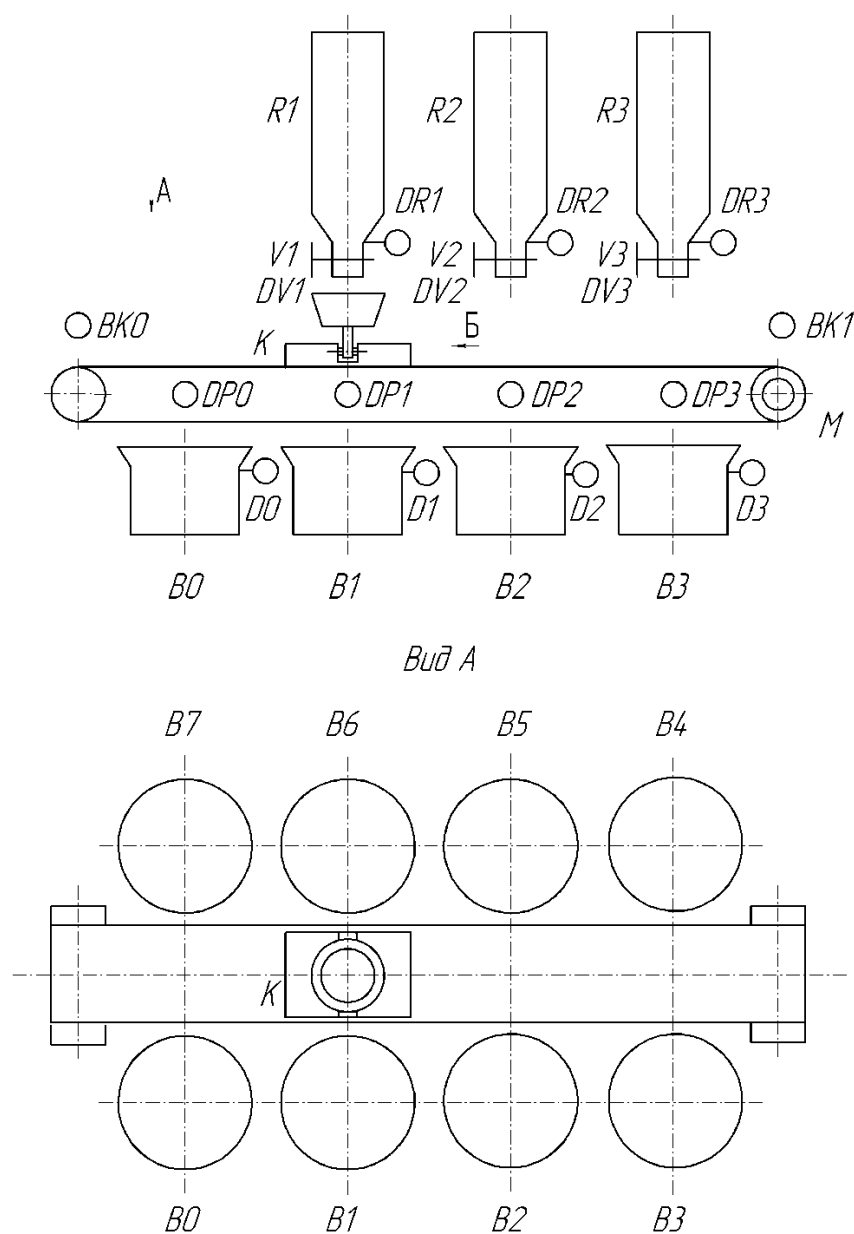


Рисунок 1 – схема расположения агрегатов распределительно-дозировочного автомата

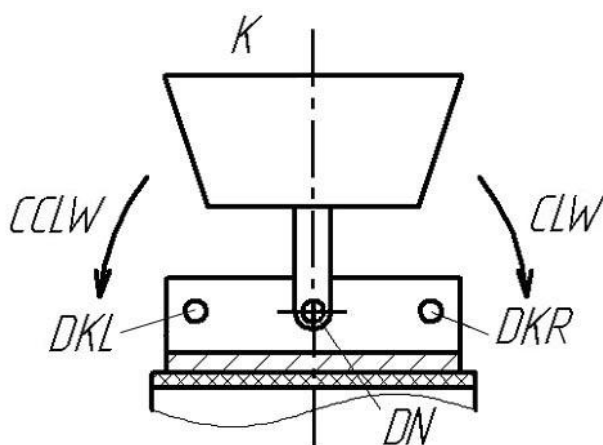


Рисунок 2 – вид на контейнер

Описание работы автомата.

Исходный материал трех видов поступает в резервуары $R1$, $R2$ и $R3$. В зависимости от задания требуется смешать эти материалы и поместить в соответствующий бункер Bj .

Смешивание происходит в контейнере K , куда материал последовательно подается из резервуаров Ri . В зависимости от задания, контейнер K подводится перемещением транспортера (от двигателя M) последовательно к соответствующим резервуаром Ri . Подвод контейнера K в заданную позицию $P0...P3$ контролируется соответствующими датчиками положения $DP0...DP3$. После подвода к резервуару Ri на заданное время открывается вентиль Vi , работа которого контролируется датчиком DVi . Контроль заполнения резервуаров и бункеров осуществляется датчиками DVi и Dj , соответственно. Датчики DVi срабатывают при отсутствии материала в резервуаре Ri , а датчики Dj срабатывают при полном заполнении бункера Bj .

Контейнер K , кроме линейных перемещений, имеет также возможность поворота (рис. 2) вокруг горизонтальной оси на $\pm 90^\circ$ для сброса материала в бункер Bj . При повороте контейнера K против часовой стрелки ($+90^\circ$) материал может сбрасываться в бункеры $B0..B3$. При повороте вдоль часовой стрелки (-90°) материал сбрасывается в бункеры $B4..B7$. Контроль поворота контейнера K при сбросе материала в бункеры Bj осуществляется тремя

датчиками: *DKR* (поворот вдоль часовой стрелки на 90^0), *DKL* (поворот против часовой стрелки на 90^0), и *DN* (исходное горизонтальное положение контейнера).

Таким образом, задачей проектирования автомата является реализация кода управления. Описание переменных (информационных и управляющих сигналов) сведено в таблицу 2.

Структура и примерное содержание КП/КР по автоматному программированию приведены в таблице 3.

Таблица 1. Кодирование исходных данных

Команда кода управления	Адреса резервуаров	Адрес бункера
000	после любого адреса ¹	<i>B0</i>
001	<i>R1</i>	<i>B1</i>
010	<i>R2</i>	<i>B2</i>
011	<i>R1, R2</i>	<i>B3</i>
...		
111	<i>R1, R2, R3</i>	<i>B7</i>

Таблица 2. Обозначения переменных

Тип переменной	Название переменной	Обозначение переменной
Информационные сигналы	Сигнал от датчика опустошения резервуара R_i ($i = 1, 2, 3$)	DR_i
	Сигнал от датчика открытия клапана V_i ($i = 1, 2, 3$)	DV_i
	Сигнал от датчика положения конвейера P_i ($i = 0 \dots 3$)	DP_i
	Сигнал от датчика при повороте контейнера на $+90^0$	DKL
	Сигнал от датчика при повороте контейнера на -90^0	DKR
	Сигнал от датчика при нормальном (вертикальном) положении контейнера	DN
	Сигнал от датчика заполнения бункера B_j ($j = 0 \dots 7$)	D_j
	Аварийные сигналы о выходе контейнера за заданные пределы	$BK0, BK1$
	Сигнал от таймера заполнения контейнера	DT
	Аварийный сигнал от таймера превышения ожидаемого времени операции	AT
Управляющие сигналы	Сигнал открытия задвижки резервуара	V_i
	Сигнал закрытия задвижки резервуара	$\overline{V_i}$
	Сигнал управления транспортером вправо	MR
	Сигнал управления транспортером влево	ML
	Аварийный сигнал остановки транспортера	$\overline{STOP}$
	Сигнал поворота контейнера вправо	MKR
	Сигнал поворота контейнера влево	MKL

¹ Команда 000 означает, что контейнер *K* заполнен неправильно (например, не хватило смешиваемого материала) и содержимое контейнера *K* необходимо отправить в бункер с отходами *B0*.

Выполним шифрование управляющей последовательности сигналов.

Таблица 3 – шифрование управляющей последовательности сигналов

ФИО студента	Р	я	з	а	н	о	в	Т	и	м	о	Ф
Код управления	1	0	1	0	1	0	1	1	0	1	0	1

Получим 4 слова по три цифры: 101 010 110 101

Команда «101» указывает, что подача будет происходить из резервуаров R_1 и R_3 , команда «110» указывает, что подача будет происходить из резервуаров R_2 и R_3 .

Номера бункеров, в которые материал подается из контейнера К: «010» - B_2 , «101» - B_5 .

Цикл 1 – берем материал из резервуаров R_1 и R_3 , смешиваем и подаем в бункер B_2

Цикл 2 – берем материал из резервуаров R_2 и R_3 , смешиваем и подаем в бункер B_5

Опишем работу в цикле 1, учитывая все аварийные ситуации:

Контейнер К находится в позиции DP_0 , двигатель выключен. При движении к резервуару R_1 включается двигатель, который двигает контейнер к R_1 , при этом $MR=1$, контейнер движется до тех пор, пока контейнер не окажется под резервуаром R_1 (пока DP_1 не будет равно 1). При достижении положения под резервуаром R_1 (при $DP_1=1$) необходимо выключить двигатель ($MR=0$), а также начать подачу материала из резервуара R_1 (перевести сигнал V_1 в 1), при этом подавать материал необходимо до тех пор, пока в контейнер не будет заполнен до необходимого предела ($DT=1$), тогда необходимо закрыть резервуар ($V_1=0$), при этом необходимо проконтролировать правильное закрытие резервуара ($DV_1=0$)

Затем контейнер К необходимо передвинуть контейнер К к резервуару R_3 , для этого мы включаем двигатель и начинаем передвигать наш контейнер вправо ($MR=1$) до тех пор, пока контейнер не достигнет резервуара R_3 ($DP_3=1$), тогда необходимо выключить двигатель ($MR=0$) и начать подавать материал из

резервуара R3 ($V3=1$). По истечению необходимого времени ($DT=1$) необходимо закрыть резервуар R3 ($V3=0$) и переместить контейнер в необходимый бункер (B2).

Для этого необходимо включить двигатель и начать двигать контейнер влево ($ML=1$) до бункера B2 ($DP2=1$). При достижении этого положения нам необходимо сбросить полученный материал в бункер, для этого необходимо повернуть контейнер по часовой стрелке ($DKR=1$, $DN=0$) и вновь поставить контейнер в горизонтальное положение ($DN=1$, $DKR=0$).

Аварийные ситуации: в резервуаре R1 или R3 может не оказаться нужного материала, или его может не хватить для корректного выполнения цикла ($DV1=1/DV3=1$), в таком случае система переходит в аварийное состояние ($AT=1$), тогда содержимое контейнера необходимо опустошить в бункер B0 (бункер с отходами), выполняется операция (дописать позднее).

Может оказаться, что в контейнере оказалось слишком мало или слишком много материала (таймер сработал некорректно), тогда материал необходимо сбросить в бункер B0.

Также может оказаться, что бункер B2 полностью заполнен ($D2=1$). В таком случае необходимо опустошить бункер B2, система переходит в аварийное состояние ($AT=1$).

Если контейнер вышел за границы конвейера ($BK0=1$ или $BK1=1$), то система автоматически останавливает конвейер ($STOP=1$) и система переходит в аварийное состояние ($AT=1$).

Опишем работу в цикле 2, учитывая все аварийные ситуации.

Контейнер после выполнения находится под резервуаром R2 ($DR2=1$), его необходимо заполнить материалом из R2. Для этого открывается резервуар ($V2=1$), пока не истечет время ($DT=1$), после этого резервуар закрывается ($V2=0$) и контейнер перемещается к резервуару R3.

Для этого необходимо включить двигатель и переместить контейнер к резервуару R3 (пока $DP3=1$). При достижении R3 двигатель выключается и контейнер заполняется материалом из R3 ($V3=1$). По прошествии времени

(DT=1) резервуар закрывается и контейнер необходимо переместить к бункеру B5.

Для этого необходимо включить двигатель и начать перемещать контейнер влево (ML=1) до бункера B5 (DP2=1). При достижении бункера, необходимо повернуть контейнер против часовой стрелки (DKL=1, DN=0), а потом вернуть бункер в изначальное положение (DN=1, DKL=0).

Аварийные ситуации: в резервуарах R2 и R3 может не оказаться материала (DR2=1/DR3=1), в таком случае система переходит в аварийное состояние (AT=1), а содержимое контейнера необходимо опустошить в бункер B0 (аналогично циклу 1).

Также может оказаться, что бункер B5 полностью заполнен (D5=1), тогда необходимо опустошить бункер B5, система переходит в аварийное состояние (AT=1).

Если контейнер вышел за границы конвейера (BK0=1 или BK1=1), то система автоматически останавливает конвейер (STOP=1) и система переходит в аварийное состояние (AT=1).

Функция сброса отходов: мы должны включить двигатель и переместить контейнер влево (ML=1), до DP=0, там мы должны сбросить отходы в бункер B0 (DKL=1), а потом возвращаем контейнер в начальное состояние (DN=1).

Введем некоторые управляющие состояния: Start – кнопка старта (запуск автомата), Reset (сброс из состояния Alarm в режим ожидания)

2 РАЗРАБОТКА ИНФОРМАЦИОННОГО ОБЕСПЕЧЕНИЯ ЗАДАЧИ

2.1 Описание конечного автомата по состояниям. Группы состояний

Запишем состояния автомата по группам (перемещение в позиции, погрузка, слив в бункеры). Это необходимо для достижения модульности, так как вызов отдельных групп команд (с состояниями) позволит легко модифицировать и изменять конечный набор команд для цикла работы автомата.

Группа S_mp0L – перемещение в позицию DP0, движение влево:

S_mp0L1: Включаем двигатель вправо, пока не достигнем DP1, иначе выключаем двигатель, если BK0==1, останавливаем движение контейнера

S_mp0L1: out (ML=1), if (DP0==1): S_mp0L1->S_mp0L2, if (BK0==1): S_mp0L1->S_bk0, if (DN==0): S_mp0L1->S_Alarm

S_mp0L2: Выключаем двигатель

S_mp0L2: out (ML=0), S_mp0L2->S_checknorm

Группа S_mp1r{} – перемещение в позицию DP1, движение вправо:

S_mp1r1: Включаем двигатель вправо, пока не достигнем DP1, иначе выключаем двигатель

S_mp1r1: out (MR=1), if (DP1==1): S_mp1r1->S_mp1r2, if (DN==0): S_mp1r1->S_Alarm

S_mp1r2: Выключаем двигатель

S_mp1r2: out (MR=0), S_mp1r2->S_checknorm

Группа S_mp1L – перемещение в позицию DP1, движение влево:

S_mp1L1: Включаем двигатель вправо, пока не достигнем DP1, иначе выключаем двигатель

S_mp1L1: out (MR=1), if (DP1==1): S_mp1L1->S_mp1L2, if (DN==0): S_mp1L1->S_Alarm

S_mp1L2: Выключаем двигатель

S_mp1L2: out (MR=0), S_mp1L2->S_checknorm

Группа S_mp2r{} – перемещение в позицию DP2, движение вправо:

S_mp2r1: Включаем двигатель вправо, пока не достигнем DP1, иначе
выключаем двигатель

S_mp2r1: out (MR=1), if (DP2==1): S_mp2r1->S_mp2r2, if (DN==0):
S_mp2r2->S_Alarm

S_mp2r2: Выключаем двигатель

S_mp2r2: out (MR=0), S_mp2r2->S_checknorm

Группа S_mp2L – перемещение в позицию DP1, движение влево:

S_mp2L1: Включаем двигатель вправо, пока не достигнем DP1, иначе
выключаем двигатель

S_mp2L1: out (MR=1), if (DP2==1): S_mp2L1->S_mp2L2, if (DN==0):
S_mp2L1->S_Alarm

S_mp2L2: Выключаем двигатель

S_mp2L2: out (MR=0), S_mp1r2->S_checknorm

Группа S_mp3r – перемещение в позицию DP3, движение вправо:

S_mp3r1: Включаем двигатель вправо, пока не достигнем DP1, иначе
выключаем двигатель, если BK0==1, останавливаем движение контейнера

S_mp3r1: out (MR=1), if (DP3==1): S_mp3r1->S_mp3r2, if (BK1==1):
S_mp3r1->S_bk1, if (DN==0): S_mp3r1->S_Alarm

S_mp3r2: Выключаем двигатель

S_mp3r2: out (MR=0), S_mp3r2->S_checknorm

**Группа S_r1 – подача материала из резервуара R1, для выполнения
необходимо выполнить группу команд S_mp1*, *-r/L (либо подход слева,
либо подход справа)**

S_r11 – открытие клапана V1, при этом V1=1 пока DV1==1 то переходим в
S_r12, если AT=1 ->S_Alarm

S_r11: V1=1, if (DV1==1): S_r11->S_r12, if (AT=1): S_r11->S_Alarm

S_r12: начинается подача материала в контейнер, таймер запущен (DT_IN=0), если DT_IN=1, то переходим в S_r13, если AT=1 то переходим в Alarm, если DR1==1, то переходим в сброс отходов (S_Rub)

S_r12: out (DT_IN=0), if (DT_IN==1): S_r12->S_r13, if (AT==1): S_r12->S_Alarm, if (DR1==1): S_r12->S_Rub

S_r13: закрытие клапана резервуара R1 (V1=0), ждем (DV1==0), далее переходим в S_r14

S_r13: out(V1=0), if (DV1==0): S_r13->S_r14, if (AT==1): S_r13->S_checknorm

Группа S_r2 – подача материала из резервуара R2, для выполнения необходимо выполнить группу команд S_mp2*, *-r/L (либо подход слева, либо подход справа)

S_r21 – открытие клапана V2, при этом V2=1 пока DV2==1 то переходим в S_r22, если AT=1 ->S_Alarm

S_r21: out(V2=1), if (DV2==1): S_r21->S_r22, if (AT=1): S_r21->S_Alarm

S_r22: начинается подача материала в контейнер, таймер запущен (DT_IN=0), если DT_IN=1, то переходим в S_r23, если AT=1 то переходим в Alarm, если DR2==1, то переходим в сброс отходов (S_Rub)

S_r22: out (DT=0), if (DT==1): S_r22->S_r23, if (AT==1): S_r22->S_Alarm, if (DR2==1): S_r22->S_Rub

S_r23: закрытие клапана резервуара R2 (V2=0), ждем (DV2==0), далее переходим в S_r24

S_r23: out (V2=0), if (DV2==0): S_r23->S_r24, if (AT==1): S_r23->S_checknorm

Группа S_r3 – подача материала из резервуара R3 для выполнения необходимо выполнить группу команд S_mp3L (либо подход слева, либо подход справа)

S_r31 – открытие клапана V3, при этом V3=1 пока DV3==1 то переходим в S_r32, если AT=1 ->S_Alarm

S_r31: out(V3=1), if (DV3==1): S_r21->S_r22, if (AT=1): S_r21->S_Alarm

S_r32: начинается подача материала в контейнер, таймер запущен (DT_IN=0), если DT_IN=1, то переходим в S_r33, если AT=1 то переходим в Alarm, если DR2==1, то переходим в сброс отходов (S_Rub)

S_r32: out (DT_IN=0), if (DT_IN==1): S_r32->S_r33, if (AT==1): S_r32->S_Alarm, if (DR2==1): S_r32->S_Rub

S_r33: закрытие клапана резервуара R3 (V3=0), ждем (DV3==0), далее переходим в S_r34

S_r33: out (V3=0), if (DV3==0): S_r33->S_r34, if (AT==1): S_r33->S_checknorm

Группа S_b2 – слив в бункер B2, для выполнения необходимо выполнить группа команд S_mp2* (либо подход слева, либо подход справа)

S_b21: проверяем, что бункер B2 может принять материал, если нет, то переходим в состояние S_Idle_Drain2

S_b21: if (D2==1): S_b21->S_Idle_Drain2, else: S_b21->S_b22

S_b22: включаем поворот вправо (MKR=1), когда DKR==1, то переходим в S_b53, если AT==1, то включаем аварийную ситуацию

S_b22: out (MKL=1), if (DKL==1): S_b22->S_b23, if (AT==1): S_b22->S_Alarm

S_b23: выключаем двигатель и сразу переходим в S_b24

S_b23: out (MKL=0), S_b23->S_b24

S_b24: ставим ожидание (DT_OUT==0), когда DT_OUE==1, переходим в S_b25

S_b24: out (DT_OUT=0), if (DT_OUT==1): S_b24->S_b25

S_b25: возвращаем контейнер в изначальное состояние (MKR=1), пока контейнер не займет нормальное положение (DN=1)

S_b25: out (MKR=1), S_b25->S_checknorm (если DN==1, то произойдет переход в S_b26)

S_b26: выключаем двигатель поворота (MKR=0)

S_b26: out (MKR=0), S_b26-> any, any – требуемое состояние для продолжения цикла

Группа S_b5 – слив в бункер B5, для выполнения необходимо выполнить группа команд S_mp2* (либо подход слева, либо подход справа)

S_b51: проверяем, что бункер B5 может принять материал, если нет, то переходим в состояние S_Idle_Drain5

S_b51: if (D5==1): S_b51->S_Idle_Drain5, else: S_b51->S_b52

S_b52: включаем поворот вправо (MKR=1), когда DKR==1, то переходим в S_b53, если AT==1, то включаем аварийную ситуацию

S_b52: out (MKR=1), if (DKR==1): S_b52->S_b53, if (AT==1): S_b52->S_Alarm

S_b53: выключаем двигатель и сразу переходим в S_b54

S_b53: out (MKR=0), S_b53->S_b54

S_b54: ставим ожидание (DT==0), когда DT==1, переходим в S_b55

S_b54: out (DT=0), if (DT==1): S_b54->S_b55

S_b55: возвращаем контейнер в изначальное состояние (MKL=1), пока контейнер не займет нормальное положение (DN=1)

S_b55: out (MKL=1), S_b55->S_checknorm (если DN==1, то произойдет переход в S_b56

S_b56: выключаем двигатель поворота (MKL=0)

S_b56: out (MKL=0), S_b56-> any, any – требуемое состояние для продолжения цикла

Группа S_Idle_Drain – контейнер с материалом ожидает на позиции, пока бункер не опустошится

S_Idle_Drain_Rub – бункер для отходов полный (D0=1), ожидаем, пока бункер опустошится, и сливаем отходы

S_Idle_Drain_Rub: if (D0=0):S_Idle_Drain_Rub->S_Rub2

S_Alarm – аварийная ситуация, в эту группу можно прийти из любых остальных групп при критической ситуации в АСУ, выход из нее осуществляется в состояние S_Idle

S_Alarm: if (Reset ==1): S_Alarm ->S_Idle

Группа S_Rub – группа сброса мусора (в бункер B0), необходимо перенести контейнер в DP0 (выполнить группу S_mp0L)

S_Rub->S_mp0L {}->S_Rub1

S_Rub1: проверяем, что бункер B0 не полный (D0=0), если он полный, то переходим в S_Idle_Drain_Rub

S_Rub1: if (D0==1): S_Rub1->S_Idle_Drain_Rub, else: S_Rub1->S_Rub2

S_Rub2: поворачиваем контейнер по часовой стрелке (MKL=1), когда DKL==1, то переходим в S_Rub3, если AT==1, то переходим в S_Alarm

S_Rub2: out (MKL=1), if (DKL==1): S_Rub2->S_Rub3, if (AT==1):S_Rub2->S_Alarm

S_Rub3: выключаем двигатель и сразу переходим в S_Rub4

S_Rub3: out (MKL=0), S_Rub3->S_Rub4

S_Rub4: ставим ожидание (DT=0), когда DT==1, то переходим в S_Rub5

S_Rub4: out (DT=0, MKL=0), if (DT==1): S_Rub4->S_Rub5

S_Rub5: поворачиваем контейнер против часовой стрелки (MKR=1), когда DN==1, переходим в S_Rub6

S_Rub5: out (MKR=1), S_Rub5->S_checknorm (если DN==1, то произойдет переход в S_Rub6)

S_Rub6: выключаем двигатель поворота

S_Rub6: out (MKR=0), S_Rub6->any, any – требуемое состояние для продолжения цикла

Группа S_checknorm – проверка контейнера на горизонтальное положение

S_checknorm: if (DN==1): S_checknorm->S_any, else: S_checknorm->S_Alarm

2.2 Таблица переходов. Граф переходов

Составим таблицу переходов:

Таблица 4 – таблица переходов управляющего автомата

Состояние	Условие	Выход	След. состояние	Пояснение
S_Idle	Start=1&DN=1&DP0=1		S_mp1r1	Старт выполн.цикла
S_Alarm	Reset=1		S_mp0L	Выход из аварийной ситуации (необходимо перевести в DP0)
S_checknorm	DN==1		S_any	
	DN==0		S_Alarm	
S_mp0L1	DP0=1	ML=1	S_mp1r2	Начало движения к DP0
	DN=0		S_Alarm	Авария (при перемещении контейнер накренился)
S_mp0L2	BK0=0	ML=0	S_checknorm	Прибытие на DP0 (слева)
	BK0=1		S_bk0	Пересечение безопасн.границы
S_bk0		STOP	S_Alarm	Остановка конвейера и переход в аварийную ситуацию
S_mp1r1	DP1=1	MR=1	S_mp1r2	Начало движения к R1
	DN=0		S_Alarm	Авария (контейнер накренился)
S_mp1r2		MR=0	S_checknorm	Отправление на DP1 (справа)
S_mp1L1	DP1=1	ML=1	S_mp1L2	Прибытие на DP1 (слева)
	DN=0		S_Alarm	Авария (контейнер накренился)
S_mp1L2		ML=0	S_checknorm	Проверка на горизонтальность контейнера
S_mp2r1	DP2=1	MR=1	S_mp2r2	Начало движения к DP1
S_mp2r2		MR=0	S_checknorm	Прибытие к DP1 (справа)
S_mp2L1	DP2=1	ML=1	S_mp2L2	Отправление на DP1 (слева)
	DN=0		S_Alarm	Авария (контейнер накренился)

S_mp2L2		ML=0	S_checknorm	Прибытие к DP2 (слева)
S_mp3r1	DP3=1	MR=1	S_mplr2	Начало движения к DP3 (справа)
	DN=0		S_Alarm	Авария (контейнер накренился)
S_mp3r2	BK1=0	MR=0	S_checknorm	Прибытие к DP3
	BK1=1		S_bk1	Пересечение безопасн.границы (справа)
S_bk1		STOP	S_Alarm	Остановка конвейера и переход в аварийную ситуацию
S_r11	DV1=1	V1=1	S_r12	Открытие клапана R1
S_r12	DT_IN=1	DT_IN=0	S_r13	Успешное нап. из R1
	DR1==1		S_Rub	Материала не хватило для корректного смешивания, необходим сброс в мусор
S_r13	DV1=0	V1=0	S_checknorm	Открытие клапана резервуара
	AT=1		S_Alarm	Процесс происходил некорректное кол-во времени
S_r21	DV2=1	V2=1	S_r22	Открытие клапана R1
S_r22	DT_IN=1	DT_IN=0	S_r23	Успешное нап. из R1
	DR2==1		S_Rub	Материала не хватило для корректного смешивания, необходим сброс в мусор
S_r23	DV2=0	V2=0	S_checknorm	Закрытие резервуара
	AT=1		S_Alarm	Неправильное закрытие резервуара
S_r31	DV1=1	V3=1	S_r12	Открытие клапана R1
S_r32	DT_IN=1	DT_IN=0	S_r13	Успешное нап. из R1

	DR3==1		S_Rub	Материала не хватило для корректного смешивания, необходим сброс в мусор
S_r33	DV1=0	V3=0	S_checknorm	Закрытие резервуара
	AT=1		S_Alarm	Неправильное закрытие резервуара
S_b21	D2==0		S_b22	Бункер не полон, можно наливать материал
	D2==1		S_Idle_Drain2	Бункер полон, ждем опустошения
S_Idle_Drain2	D2==0		S_b22	Бункер опустошен, можно наливать материал
S_b22	DKL=1	MKL=1	S_b23	Поворот контейнера влево
	AT==1		S_Alarm	
S_b23		MKL=0	S_b24	Выключение двигателя поворота
S_b24	DT_OUT=1	DT_OUT=0	S_b25	Ожидание, пока контейнер опустошится
S_b25	DN==1	MKR=1	S_b26	
S_b26		MKR=0	S_any	Выключение двигателя поворота
S_b51	D5==0		S_b52	Бункер не полон, можно наливать материал
	D5==1		S_Idle_Drain5	Бункер полон, ждем опустошения
S_Idle_Drain5	D5==0		S_b52	Бункер опустошен, можно наливать материал
S_b52	DKR=1	MKR=1	S_b53	Поворот контейнера вправо
	AT==1		S_Alarm	
S_b53		MKR=0	S_b54	Выключение поворота контейнера
S_b54	DT_OUT=1	DT_OUT=0	S_b55	Ожидание, пока контейнер опустошится
S_b55	DN==1	MKL=1	S_b56	Выключение двигателя поворота
S_b56		MKL=0	S_any	

S_Rub1	D0==0		S_Rub2	Бункер не полон, можно наливать материал
	D0==1		S_Idle_Drain_Rub	Бункер полон, ждем опустошения
S_Idle_Drain_Rub	D0==0		S_Rub2	Бункер опустошен, можно наливать материал
S_Rub2	DKL=1	MKL=1	S_Rub3	Поворот контейнера влево
	AT==1		S_Alarm	
S_Rub3		MKL=0	S_Rub4	Выключение двигателя поворота
S_Rub4	DT_OUT=1	DT_OUT=0	S_Rub5	Ожидание, пока контейнер опустошится
S_Rub5	DN==1	MKR=1	S_Rub6	
S_Rub6		MKR=0	S_any	Выключение двигателя поворота

Составим граф переходов для групп состояний:

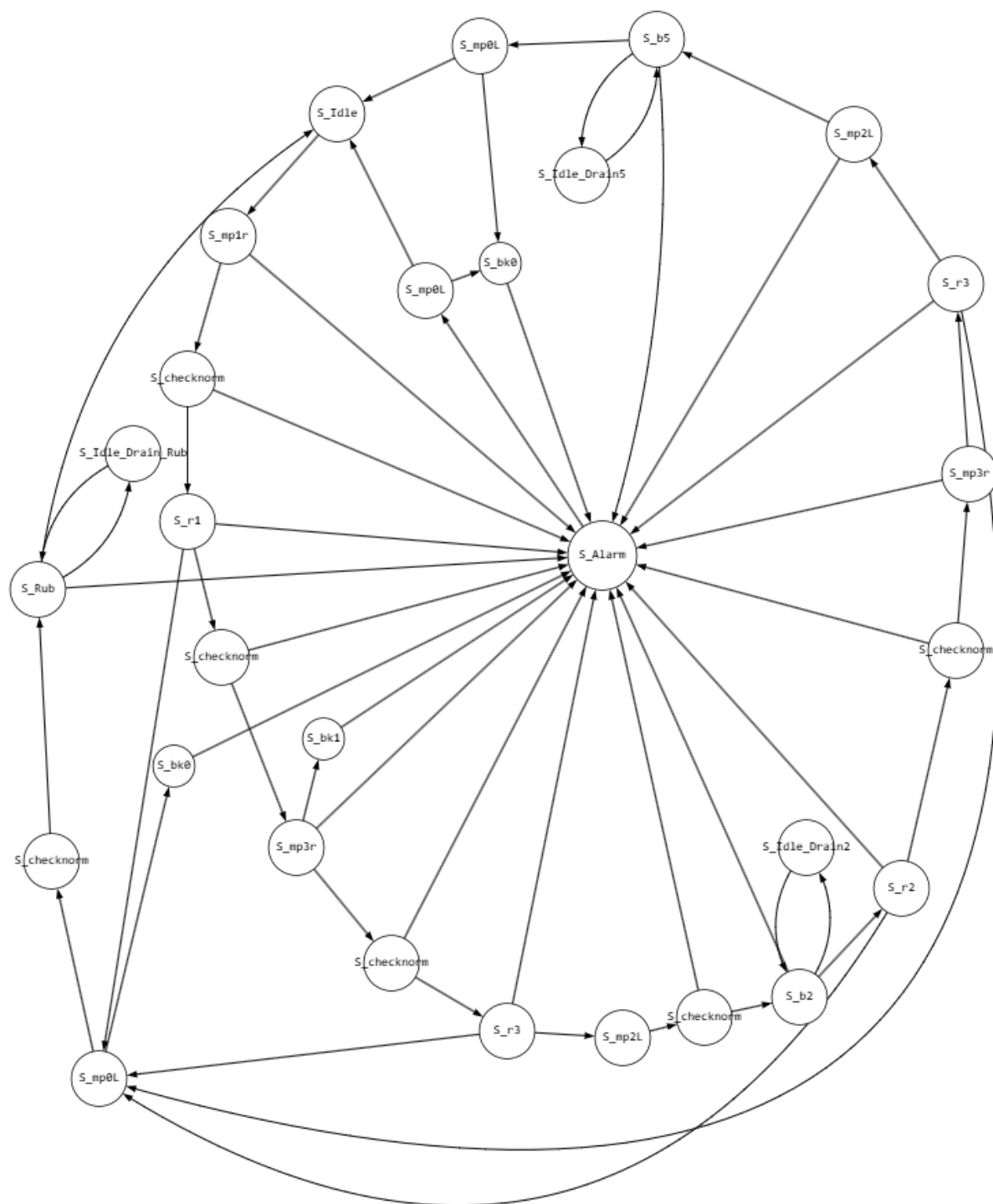


Рисунок 3 – граф переходов для групп состояний

2.3 Минимизация конечного автомата

Минимизация выходов. Составим таблицу состояний и их двоичную кодировку:

Таблица 5 – двоичная шифровка состояний

Состояние	Двоичный код (q5, q4, q3, q2, q1, q0)
S_Idle	000000
S_Alarm	000001
S_mp0L1	000010

S mp0L2	000011
S bk0	000100
S mp1r1	000101
S mp1r2	000110
S mp1L1	000111
S mp1L2	001000
S mp2r1	001001
S mp2r2	001010
S mp2L1	001011
S mp2L2	001100
S mp3r1	001101
S mp3r2	001110
S bk1	001111
S r11	010000
S r12	010001
S r13	010010
S r21	010011
S r22	010100
S r23	010101
S r31c1	010110
S r32c1	010111
S r33c1	011000
S b21	011001
S Idle Drain2	011010
S b22	011011
S b23	011100
S b24	011101
S b25	011110
S b26	011111
S b51	100000
S Idle Drain5	100001
S b52	100010
S b53	100011
S b54	100100
S b55	100101
S b56	100110
S Rub1	100111
S Idle Drain Rub	101000
S Rub2	101001
S Rub3	101010
S Rub4	101011
S Rub5	101100
S Rub6	101101
S checknorm	101110
S r31c2	101111
S r32c2	110000
S r33c2	110001
S mp2L12	110010
S mp2L22	110011
S mp2L13	110100

S mp2L23	110101
S mp0LR1	110110
S mp0LR2	110111
S mp0LI1	111000
S mp0LI2	111001

Распишем выходы через состояния:

$$ML =$$

$$S_mp0L1 + S_mp1L1 + S_mp2L1 + S_mp2L12 + S_mp2L13 + S_mp0LR1 + S_mp0LI1$$

$$\neg ML =$$

$$S_mp0L2 + S_mp1L2 + S_mp2L2 + S_mp2L22 + S_mp2L23 + S_mp0LR2 + S_mp0LI2$$

$$MR = S_mp1r1 + S_mp2r1 + S_mp3r1$$

$$\neg MR = S_mp1r2 + S_mp2r2 + S_mp3r2$$

$$STOP = S_bk0 + S_bk1$$

$$V1 = S_r11, \neg V1 = S_r13$$

$$V2 = S_r21, \neg V2 = S_r23$$

$$V3 = S_r31c1 + S_r31c2, \neg V3 = S_r33c1 + S_r33c2$$

$$\neg DT_IN = S_r12 + S_r22 + S_r32$$

$$\neg DT_OUT = S_b24 + S_b54 + S_Rub4$$

$$MKR = S_b25 + S_b52 + S_Rub5$$

$$\neg MKR = S_b26 + S_b53 + S_Rub6$$

$$MKL = S_b55 + S_b22 + S_Rub2$$

$$\neg MKL = S_b56 + S_b23 + S_Rub3$$

Запишем выходы через разряды (q5, q4, q3, q2, q1, q0):

$$ML = \neg q5 \neg q4 \neg q3 \neg q2 q1 \neg q0 + \neg q5 \neg q4 \neg q3 q2 q1 q0 +$$

$$\neg q5 \neg q4 q3 \neg q2 q1 \neg q0 + q5 q4 \neg q3 \neg q2 q1 \neg q0 + q5 q4 \neg q3 q2 \neg q1 \neg q0 + q5 q4 \neg q3 q2 q1 \neg q0 + q5 q4 q3 \neg q2 \neg q1 \neg q0$$

$$\neg ML = \neg q5 \neg q4 \neg q3 \neg q2 q1 q0 + \neg q5 \neg q4 q3 \neg q2 \neg q1 \neg q0 +$$

$$\neg q5 \neg q4 q3 q2 \neg q1 \neg q0 + q5 q4 \neg q3 \neg q2 q1 q0 + q5 q4 \neg q3 q2 \neg q1 q0 + q5 q4 \neg q3 q2 q1 q0 + q5 q4 q3 \neg q2 \neg q1 q0$$

$$MR = \neg q5 \neg q4 \neg q3 q2 \neg q1 q0 + \neg q5 \neg q4 q3 \neg q2 \neg q1 q0 + \neg q5 \neg q4 q3 q2 \neg q1 q0$$

$$\neg MR = \neg q5 \neg q4 \neg q3 q2 q1 \neg q0 + \neg q5 \neg q4 q3 \neg q2 q1 \neg q0 + \neg q5 \neg q4 q3 q2 q1 \neg q0$$

$$\text{STOP} = \neg q5 \neg q4 \neg q3 q2 \neg q1 \neg q0 + \neg q5 \neg q4 q3 q2 q1 q0$$

$$V1 = \neg q5 q4 \neg q3 \neg q2 \neg q1 \neg q0$$

$$\neg V1 = \neg q5 q4 \neg q3 \neg q2 q1 \neg q0$$

$$V2 = \neg q5 q4 \neg q3 \neg q2 q1 q0$$

$$\neg V2 = \neg q5 q4 \neg q3 q2 \neg q1 q0$$

$$V3 = \neg q5 q4 \neg q3 q2 q1 \neg q0 + q5 \neg q4 q3 q2 q1 q0$$

$$\neg V3 = \neg q5 q4 q3 \neg q2 \neg q1 \neg q0 + q5 q4 \neg q3 \neg q2 \neg q1 q0$$

$$\neg \text{DT_IN} = \neg q5 q4 \neg q3 \neg q2 \neg q1 q0 + \neg q5 q4 \neg q3 q2 \neg q1 \neg q0 + \neg q5 q4 \neg q3 q2 q1 q0$$

$$\neg \text{DT_OUT} = \neg q5 q4 q3 q2 \neg q1 q0 + q5 \neg q4 \neg q3 q2 \neg q1 \neg q0 + q5 \neg q4 q3 \neg q2 q1 q0$$

$$\text{MKR} = \neg q5 q4 q3 q2 q1 \neg q0 + q5 \neg q4 \neg q3 \neg q2 q1 \neg q0 + q5 \neg q4 q3 q2 \neg q1 \neg q0$$

$$\neg \text{MKR} = \neg q5 q4 q3 q2 q1 q0 + q5 \neg q4 \neg q3 \neg q2 q1 q0 + q5 \neg q4 q3 q2 \neg q1 q0$$

$$\text{MKL} = \neg q5 q4 q3 \neg q2 q1 q0 + q5 \neg q4 \neg q3 q2 \neg q1 q0 + q5 \neg q4 q3 \neg q2 \neg q1 q0$$

$$\neg \text{MKL} = \neg q5 q4 q3 q2 \neg q1 \neg q0 + q5 \neg q4 \neg q3 q2 q1 \neg q0 + q5 \neg q4 q3 \neg q2 q1 \neg q0$$

$$\text{Alarm} = S_Alarm$$

Как видно выше, в минимизации участвуют не все состояния, так как некоторые состояния не имеют выхода.

Для минимизации выходов воспользуемся законами алгебры логики:

- 1) $A \neg A = 0, A + \neg A = 1$
- 2) $A * 1 = A, A + 0 = A$
- 3) $A(A+B) = A, A + AB = A$
- 4) $A+BC = (A+B)(B+C)$
- 5) $(AB)+(A\neg B) = A, (A+B)(A+\neg B) = A$

В ходе упрощений получим:

$$\text{ML} \quad \quad \quad =$$

$$\neg q5 \neg q4 \neg q2 q1 \neg q0 + q5 q4 \neg q3 q1 \neg q0 + q5 q4 \neg q3 q2 \neg q0 + \neg q5 \neg q4 \neg q3 q2 q1 q0 + q5 q4 q3 \neg q2 \neg q1 \neg q0$$

$$\neg \text{ML} \quad \quad \quad =$$

$$q5 q4 \neg q3 q1 q0 + q5 q4 \neg q3 q2 q0 + \neg q4 \neg q4 q3 \neg q1 \neg q0 + \neg q5 \neg q4 \neg q3 \neg q2 q1 q0 + q5 q4 q3 \neg q2 \neg q1 q0$$

$$\text{MR} = \neg q5 \neg q4 \neg q3 \neg q1 q0 + \neg q5 \neg q4 q2 \neg q1 q0$$

$$\begin{aligned}
\neg MR &= \neg q5 \neg q4 q2 q1 \neg q0 + \neg q5 \neg q4 q3 q1 \neg q0 \\
STOP &= \neg q5 \neg q4 \neg q3 q2 \neg q1 \neg q0 + \neg q5 \neg q4 q3 q2 q1 q0 \\
V1 &= \neg q5 q4 \neg q3 \neg q2 \neg q1 \neg 10 \\
\neg V1 &= \neg q5 q4 \neg q3 \neg q2 q1 \neg q0 \\
V2 &= \neg q5 q4 \neg q3 \neg q2 q1 q0 \\
\neg V2 &= \neg q5 q4 \neg q3 q2 \neg q1 q0 \\
V3 &= \neg q5 q4 \neg q3 q2 q1 \neg q0 + q5 \neg q4 q3 q2 q1 q0 \\
\neg V3 &= \neg q5 q4 q3 \neg q2 \neg q1 \neg q0 + q5 q4 \neg q3 \neg q2 \neg q1 q0 \\
\neg DT_IN &= \neg q5 q4 \neg q3 \neg q2 \neg q1 q0 + \neg q5 q4 \neg q3 q2 \neg q1 \neg q0 + \neg q5 q4 \neg q3 q2 q1 q0 \\
\neg DT_OUT &= \neg q5 q4 q3 q2 \neg q1 q0 + q5 \neg q4 \neg q3 q2 \neg q1 \neg q0 + q5 \neg q4 q3 \neg q2 q1 q0 \\
MKR &= \neg q5 q4 q3 q2 q1 \neg q0 + q5 \neg q4 \neg q3 \neg q2 q1 \neg q0 + q5 \neg q4 q3 q2 \neg q1 \neg q0 \\
\neg MKR &= \neg q5 q4 q3 q2 q1 q0 + q5 \neg q4 \neg q3 \neg q2 q1 q0 + q5 \neg q4 q3 q2 \neg q1 q0 \\
MKL &= \neg q5 q4 q3 \neg q2 q1 q0 + q5 \neg q4 \neg q3 q2 \neg q1 q0 + q5 \neg q4 q3 \neg q2 \neg q1 q0 \\
\neg MKL &= \neg q5 q4 q3 q2 \neg q1 \neg q0 + q5 \neg q4 \neg q3 q2 q1 \neg q0 + q5 \neg q4 q3 \neg q2 q1 \neg q0 \\
Alarm &= S_Alarm
\end{aligned}$$

3 РАЗРАБОТКА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ ЗАДАЧИ

3.1. Реализация на языке C++

В ходе курсовой работы также была проведена программная реализация конечного автомата.

Для написания программной реализации был выбран язык C++, так как он известен как быстродействующий язык, что критически важно для автоматизированных систем управления, ведь малейшая задержка может привести к финансовым потерям или даже ранениям рабочих.

Достоинства языка C++:

- 1) Высокая производительность — язык компилируется в машинный код, который может быть выполнен непосредственно процессором. Это делает C++ подходящим для разработки высоконагруженных систем.
- 2) Поддержка объектно-ориентированного программирования (ООП): инкапсуляция, наследование и полиморфизм. Это позволяет структурировать код так, чтобы он был легко расширяемым и поддерживаемым.
- 3) Широкие возможности низкоуровневого программирования — программист может управлять памятью напрямую через указатели и другие механизмы. Это позволяет оптимизировать работу приложения под конкретные аппаратные ресурсы, что особенно важно в системах реального времени и встроеном ПО.
- 4) Кроссплатформенность — код на C++ можно компилировать под различные платформы (Windows, Linux, macOS и др.).
- 5) Большой выбор библиотек — они могут использоваться для решения различных задач, включая многопоточность, работу с сетью, математические вычисления.

Недостатки использования языка C++:

1) Сложность изучения и использования — язык требует глубокого понимания многих концепций, таких как управление памятью, работа с указателями и шаблонами. Новичкам может быть трудно освоить эти концепции, особенно если они привыкли работать с более высокоуровневыми языками.

2) Необходимость ручного управления памятью — программист должен самостоятельно выделять и освобождать память, что увеличивает риск возникновения утечек памяти и других ошибок.

3) Долгое время компиляции — из-за сложности синтаксиса и необходимости обработки большого количества метаданных компиляторы C++ часто работают медленнее, чем компиляторы других языков. Это может замедлить цикл разработки, особенно при работе над крупными проектами.

4) Отсутствие стандартной библиотеки для некоторых современных задач — хотя стандартная библиотека C++ содержит множество полезных функций и классов, она не всегда удовлетворяет потребности современных разработчиков. Например, в ней отсутствуют готовые решения для веб-разработки или работы с базами данных, что вынуждает программистов обращаться к сторонним библиотекам.

Так как в пункте 2 автоматная логика была реализована через группы состояний, напрашивается необходимость использования ООП. Для этого в программной реализации были реализованы классы групп состояний. В них уже прописана вся необходимая логика, и, при необходимости, классы могут быть дополнены, а также в код могут быть добавлены новые классы, что является преимуществом при необходимости расширения или модификации имеющейся системы.

Системные требования для языка C++:

Процессор (CPU): 2-ядерный, с частотой от 1.8 ГГц (например, Intel Core i3 5-го поколения или новее, AMD A-series).

Оперативная память (RAM): 4 ГБ — абсолютный минимум, но возможны подтормаживания. 8 ГБ — комфортный минимум.

Жесткий диск (Storage): 50 ГБ свободного места (HDD приемлем, но SSD предпочтительнее).

Операционная система (OS):

- 1) Windows: 10 (64-bit)
- 2) Linux: Ubuntu 20.04 LTS / Fedora 36 или новее
- 3) macOS: Monterey (12) или новее

Рекомендуемые IDE:

- 1) Visual Studio Code (с расширениями C++, CMake)
- 2) Code::Blocks
- 3) Qt Creator (в базовой конфигурации)

Код, реализующий управляющий автомат, а также имитирующий реальное поведение дозировочного автомата представлен в листинге 1.

Так как в части 2 работы были реализованы группы состояний, то в коде используются принципы ООП, а классы представляют собой эти группы состояний. В каждом классе реализованы переходы в состояния внутри группы, а также переходы в другие необходимые состояния. В начале реализованы структуры информационных и управляющих сигналов, к которым в течении всего кода происходит обращение. Для имитации реальной работы автомата используется функция `SimulatePhysics`, которая выжидает некоторое количество времени и обновляет состояние автомата.

Вся имитация работы, в частности, обновление значений датчиков и обновление автомата, реализованы в функции `main`. В автомате реализован вывод текущего состояния, состояния резервуаров, а также перемещение контейнера в консоль, чтобы диспетчер мог отслеживать поведение автомата. Запуск из консоли осуществляется с помощью данной команды:

```
$exePath = Join-Path (Get-Location) "код для курсовой работы.exe"  
& $exePath
```

Перед тем, как запустить команду выше, необходимо открыть папку с файлом формата `.exe` через терминал. После запуска этой команды, в терминале будет следующий вывод:

```
Windows PowerShell
PS C:\Users\hara1\Desktop\учеба\теория конечных автоматов\курсач\код для курсовой работы\x64\Debug> $exePath = Join-Path (Get-Location) "код для курсовой работы.exe"
PS C:\Users\hara1\Desktop\учеба\теория конечных автоматов\курсач\код для курсовой работы\x64\Debug> & $exePath
Переход в состояние: Idle
Симуляция: Оператор нажимает кнопку START. Контейнер на базе (DP0).
Запуск цикла...
Переход в состояние: Move_to_R1_Right
Клапан резервуара R1 закрыт
Клапан резервуара R2 закрыт
Клапан резервуара R3 закрыт

Симуляция: Контейнер поехал вправо... (DP0=0)
Прибытие на позицию DP1Переход в состояние: CheckNorm
Клапан резервуара R1 закрыт
Клапан резервуара R2 закрыт
Клапан резервуара R3 закрыт

Симуляция: Прибыли к датчику DP1. Проверка горизонта (DN=1).
Переход в состояние: Filling_From_R1
Клапан резервуара R1 закрыт
Клапан резервуара R2 закрыт
Клапан резервуара R3 закрыт

Симуляция: Клапан R1 открылся (DV[0]=1).
Клапан резервуара R1 открыт
Клапан резервуара R2 закрыт
Клапан резервуара R3 закрыт

Симуляция: Бак наполнился! (DT_IN=1).
Клапан резервуара R1 закрыт
Клапан резервуара R2 закрыт
Клапан резервуара R3 закрыт

Симуляция: Клапан R1 закрылся (DV[0]=0).
Переход в состояние: Move_to_R3_Left
Клапан резервуара R1 закрыт
Клапан резервуара R2 закрыт
Клапан резервуара R3 закрыт

Симуляция: Контейнер уехал с DP1 к R3...
Прибытие на позицию DP3Переход в состояние: CheckNorm
Клапан резервуара R1 закрыт
Клапан резервуара R2 закрыт
```

Рисунок 4 – запуск кода управляющего автомата

Это и есть выполнение кода, который представлен в листинге 1. Как видно, диспетчеру видна некоторая информация об автомате.

3.2 Реализация автомата на триггерах

Конечные автоматы можно также реализовывать на триггерах, для этого необходимо закодировать состояния, минимизировать выходы автомата и задать переключательную функцию. Первые два пункта были реализованы в части 2 курсовой работы, в пункте 2.3.

Так как в итоговом конечном автомате достаточно много состояний, а из одного состояния можно перейти сразу в несколько других состояний, то реализация без помощи специальных сложных компонентов будет достаточно проблематично. Для упрощения задачи будет использовано переключение с помощью дешифратора и шифратора. Переключательная функция будет реализована с помощью логических элементов И, ИЛИ и НЕ. Задумка состоит в том, чтобы по какому-то условию (сработал определенный датчик), сигнал с определенного выхода с дешифратора будет направлен на другой, заранее

известный выход шифратора, который будет преобразовывать номер контакта в двоичную запись.

Для корректной работы автомата добавим дополнительные состояния автомата. Это сделано для того, чтобы исправить множественные переходы в исходном автомате, так как триггеры не могут помнить, из какого состояния они прибыли в текущее состояние, для этого необходимо разделить состояния с множественными переходами. Это необходимо для того, чтобы автомат на триггерах мог корректно работать.

Реализация переключательной функции содержится в таблице 6.

Таблица 6 – переключательная функция для конечного автомата

Исходное состояние	Двоичная кодировка	№ исх.сост	Условие	Конечное состояние	Двоичная кодировка	№ кон.сост
S_Idle	000000	1	Start==1 AND DN==1 AND DP0==1	S_mp1r1	000101	5
S_Alarm	000001	2	Reset==1	S_mp0L1	111000	57
S_mp0L1	000010	3	DP0==1	S_mp0L2	000011	4
S_mp0L1	000010	3	DN==0	S_Alarm	000001	2
S_mp0L2	000011	4	BK0==1 AND DN==1	S_bk0	000100	5
S_mp0L2	000011	4	BK0==0 AND DN==1 AND Start==0	S_Idle	000000	1
S_mp0L2	000011	4	BK0==0 AND DN==1	S_mp1r1	000101	6

			AND Start==1			
S_bk0	000100	5		S_Alarm	000001	2
S_mp1r1	000101	6	DP1==1	S_mp1r2	000110	7
S_mp1r2	000110	7	DN==1	S_r11	010000	17
S_mp1L1	000111	8	DP1==1	S_mp1L2	001000	9
S_mp1L2	001000	9	DN==1	S_r11	010000	17
S_mp1L2	001000	9	DN==0	S_Alarm	000001	2
S_mp2r1	001001	10	DP2==1	S_mp2r2	001010	11
S_mp2r2	001010	11	DN==1	S_r21	010010	47
S_mp2r2	001010	11	DN==1	S_b21	011001	26
S_mp2r2	001010	11	DN==1	S_b51	100000	33
S_mp2r2	001010	11	DN==0	S_Alarm	000001	2
S_mp2L1	001011	12	DP2==1	S_mp2L2	001100	13
S_mp2L2	001100	13	DN==1	S_r21	101110	47
S_mp2L12	110010	51	DP2==1	S_mp2L22	110011	52
S_mp2L22	110011	52	DN==1	S_b21	011001	26
S_mp2L13	110100	53	DP2==1	S_mp2L23	110101	54
S_mp2L23	110101	54	DN==1	S_b51	100000	
S_mp3r1	001101	14	DP3==1	S_mp3r2	001110	15
S_mp3r2	001110	15	DN==1	S_r31	010110	23
S_mp3r2	001110	15	BK1==1	S_bk1	001111	16
S_r11	010000	17	DV1==1	S_r12	010001	18
S_r12	010001	18	DT_IN==1	S_r13	010010	19
S_r12	010001	18	DR1==1	S_mp0LR1	110110	55
S_r13	010010	19	DV1==0 AND DN==1	S_mp3r1	001101	14
S_r21	010011	20	DV2==1	S_r22	010001	18
S_r22	010100	20	DT_IN==1	S_r23	010010	19
S_r22	010100	20	DR2==1	S_mp0LR1	110110	55
S_r23	010101	21	DV2==0 AND DN==1	S_mp3r1	001101	14

S_r31c1	010110	19	DV1==0 AND DN==1	S_mp3r1	001101	14
S_r32c1	010111	17	DV2==1	S_r32c1	010001	18
S_r32c1	010111	17	DR3==1	S_mp0LR1	110110	55
S_r33c1	011000	18	DT_IN==1	S_r33c1	010010	19
S_r21	010011	19	DV1==0 AND DN==1	S_mp2L2	001101	14
S_r31c2	101111	48	DV3==1	S_r23c2	110000	49
S_r32c2	110000	49	DT_IN==1	S_r33c2	110001	50
S_r32c2	110000	49	DR3==1	S_mp0LR1	110110	55
S_r33c2	110001	50	DV3==0 AND DN==1	S_mp2L13	110100	53
S_b21	011001	26	D2==0	S_b22	011011	28
S_b21	011001	26	D2==1	S_Idle_Dran2	011010	27
S_Idle_Drain2	011010	27	D2==0	S_b22	011011	28
S_b22	011011	28	DKL==1	S_b23	011100	29
S_b22	011011	28	AT==1	S_Alarm	000001	2
S_b23	011100	29		S_b24	011101	30
S_b24	011101	30	DT_OUT=1	S_b25	011110	31
S_b25	011110	31	DN==1	S_b26	011111	32
S_b26	011111	32		S_r21	010000	17
S_b51	100000	33	D5==0	S_b52	100010	35
S_b52	100000	33	D5==1	S_Idle_Drain5	100001	34
S_Idle_Drain5	100001	34	D5==0	S_b52	100010	35
S_b52	100010	35	DKR==1	S_b53	100011	36
S_b53	100011	36		S_b54	100100	37
S_b54	100100	37	DT_OUT== 1	S_b55	100101	38
S_b55	100101	38	DN==1	S_b56	100110	39
S_b56	100110	39		S_mp0L1	000010	3
S_Rub1	100111	40	D0==0	S_Rub2	101001	42

Проанализируем работу управляющего автомата на триггерах. Шифровка состояний производится в соответствии с их двоичной записью, которые расписаны в таблице 5. Двоичная запись, которая определяет текущее состояние конечного автомата, с помощью дешифратора (на схеме представлен микросхемой Decoder) преобразуется в номер состояния, в котором на данный момент находится управляющий автомат.

С помощью схемы переключательной функции (на схеме она представлена микросхемой switchfunc) автомат реагирует на изменение условий, изменяя двоичную запись на основе комбинации информационных сигналов. Обработка происходит согласно таблице 6.

После обработки в switchfunc, текущее состояние подается на вход шифратора, где преобразуются обратно в двоичный код, который поступает на триггеры и функцию выходов.

В триггерной схеме разряды переключаются по сигналу Clock, и затем снова подаются на вход дешифратора Decoder. В функции выходов на основе двоичной записи формируется комбинация выходных сигналов. Комбинация выходных сигналов подбирается на основе минимизации выходов, которая была произведена в пункте 2.3.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1) Каширская, Е. Н. Теория конечных автоматов : учебное пособие / Е. Н. Каширская, М. М. Клягин, В. А. Серебрянкин. — Москва : РТУ МИРЭА, 2021. — 100 с. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/226538> (дата обращения: 14.12.2025). — Режим доступа: для авториз. пользователей.
- 2) Гашков, С. Б. Дискретная математика : учебник для вузов / С. Б. Гашков. — 3-е изд., испр. и доп. — Санкт-Петербург : Лань, 2025. — 520 с. — ISBN 978-5-507-49866-6. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/451232> (дата обращения: 14.12.2025). — Режим доступа: для авториз. пользователей.
- 3) Баранов, Самарий Иосифович. Синтез микропрограммных автоматов [Текст]. — Ленинград : [б. и.], 1971. — 36 с. : черт. : 21 см — (Приборы и устройства радиоэлектронной техники и автоматики/ Ленингр. организация о-ва "Знание" РСФСР. Ленингр. дом науч.-техн. пропаганды).
- 4) Харрис, Дэвид М. Цифровая схемотехника и архитектура компьютера : [пер. с англ.] / Дэвид М. Харрис, Сара Л. Харрис. — Москва : ДМК Пресс, 2017. — 791 с. : ил.; ISBN 978-5-97060-522-6.
- 5) Страуструп, Бьерн. Язык программирования C++ / Бьерн Страуструп ; пер. с англ. под ред. Н. Н. Мартынова. - Спец. изд. - Москва: Бином, 2011. - 1135 с.: ил. ; 24 см.\
- 6) Петров, Игорь Викторович Программируемые контроллеры. Стандартные языки и приемы прикладного проектирования: = Стандартные языки и приемы прикладного проектирования [Стандарт МЭК 61131. Инструменты программирования. Языки МЭК. Стандарт. компоненты. Примеры программирования] / И.В. Петров; под ред. проф. В.П. Дьяконова. — Москва: Солон-Пресс, 2004. — 253 с.: ил., табл.: 20 см — (Серия "Библиотека инженера").; ISBN 5-98003-079-4 (в обл.).

Приложение

Листинг 1 – программа, реализующая управляющий автомат

```
#include <iostream>
#include <memory>
#include <string>
#include <thread>
#include <chrono>

using namespace std;

struct Sensors {
    bool Start = false;
    bool Reset = false;
    bool DN = true;
    bool DP[4] = { false, false, false, false};
    bool DV[3] = { false };
    bool DR[3] = { false };

    bool D[8] = { false };

    bool BK0 = false;
    bool BK1 = false;

    bool DKR = false;
    bool DKL = false;

    bool AT = false;
    bool DT_IN = false;
    bool DT_OUT = false;
};

struct Controls {
    bool ML = false;
    bool MR = false;
    bool STOP = false;
    bool V[3] = { false };

    bool MKR = false;
    bool MKL = false;
    void reset() {
        ML = MR = STOP = MKL = MKR = false;
        V[0] = V[1] = V[2] = false;
    }
    void print() {
        //if (ML) {cout << "[>>] ДВИЖЕНИЕ: Контейнер едет ВЛЕВО" << endl;}
        //else if (MR) {cout << "[>>] ДВИЖЕНИЕ: Контейнер едет ВПРАВО" <<
endl;}}
        //else {cout << "[--] ДВИЖЕНИЕ: Контейнер СТОИТ на месте" <<
endl;}}

        // Логика поворота
        if (MKL) {
            cout << "Поворот контейнера: ВЛЕВО" << endl;
        }
        else if (MKR) {
            cout << "Поворот контейнера: ВПРАВО" << endl;
        }
        if (STOP) { cout << "Аварийная остановка:пересечение допустимой
границы!\n";}
        if (V[0]) { cout << "Клапан резервуара R1 открыт\n"; }
        if (!(V[0])) { cout << "Клапан резервуара R1 закрыт\n"; }
```

```

        if (V[1]) { cout << "Клапан резервуара R2 открыт\n"; }
        if (!(V[1])) { cout << "Клапан резервуара R2 закрыт\n"; }
        if (V[2]) { cout << "Клапан резервуара R3 открыт\n"; }
        if (!(V[2])) { cout << "Клапан резервуара R3 закрыт\n"; }

    }
};

class MachineContext;
class State {
public:
    virtual ~State() = default;
    virtual void enter(MachineContext* ctx) {}
    virtual void execute(MachineContext* ctx) = 0;
    virtual void exit(MachineContext* ctx) {}
    virtual string getName() const = 0;
};

class IdleState;
class AlarmState;
class CheckNormState;
class MovingToR0LefttState;
class MovingToR1LefttState;
class MovingToR1RightState;
class MovingToR2LefttState;
class MovingToR2RightState;
class FillingFromR1;
class FillingFromR2;
class FillingFromR3;
class RubbishDump;
class Bunker2_Dump;
class Bunker5_Dump;

class MachineContext {
public:
    Sensors sensors;
    Controls controls;
    unique_ptr<State> currentState;

    void changeState(State* newState);
    void update();
};

class AlarmState :public State {
public:
    void enter(MachineContext* ctx) override {
        ctx->controls.reset();
        ctx->controls.STOP = true;
    }
    void execute(MachineContext* ctx) override;
    string getName() const override { return "ALARM"; }
};

class CheckNormState :public State {
    State* nextState;
public:
    CheckNormState(State* next) : nextState(next) {}

    void execute(MachineContext* ctx) override {
        if (ctx->sensors.DN) {

```

```

        ctx->changeState(nextState);          //если контейнер
горизонтален, то переходим в следующее состояние, если нет, то переходим в
аварию
    }
    else {
        ctx->changeState(new AlarmState());
        return;
    }
}
string getName() const override { return "CheckNorm"; }
};

class MovingToR0LeftState :public State {
    int SubState = 1;
    State* nextState;
public:
    MovingToR0LeftState(State* next) : nextState(next) {}          //сюда можно
передавать как переход в ожидание, так и переход в сброс мусора

    void execute(MachineContext* ctx) override {
        switch (SubState) {
            case 1:
                ctx->controls.ML = true;
                if (!ctx->sensors.DN) {
                    cout << "[!] Аварийная ситуация! Контейнер накренился
во время передвижения!";
                    ctx->changeState(new AlarmState());
                    nextState = nullptr;
                    break;
                }
                if (ctx->sensors.DP[0]) {
                    SubState = 2;
                }
            case 2:
                ctx->controls.ML = false;
                cout << "Прибытие на позицию DP0";
                ctx->changeState(new CheckNormState(nextState));
                //успешное выполнение группы S_мр0L
                nextState = nullptr;
                break;
        }
    }
    string getName() const override { return "Move_to_R0_Left"; }
};

class MovingToR1LeftState :public State {
    int SubState = 1;
    State* nextState;
public:
    MovingToR1LeftState(State* next) : nextState(next) {}          //сюда можно
передавать как сброс в бункер B1B6, так и получение материала из R1

    void execute(MachineContext* ctx) override {
        switch (SubState) {
            case 1:
                ctx->controls.ML = true;
                if (!ctx->sensors.DN) {
                    cout << "[!] Аварийная ситуация! Контейнер накренился
во время передвижения!";
                    ctx->changeState(new AlarmState());
                    nextState = nullptr;

```

```

        break;
    }
    if (ctx->sensors.DP[1]) {
        SubState = 2;
    }
    case 2:
        ctx->controls.ML = false;
        cout << "Прибытие на позицию DP1";
        ctx->changeState(new CheckNormState(nextState));
//успешное выполнение группы S_mplL
        nextState = nullptr;
        break;
    }

}
string getName() const override { return "Move_to_R1_Left"; }
};

class MovingToR1RightState :public State {
    int SubState = 1;
    State* nextState;
public:
    MovingToR1RightState(State* next): nextState(next){} //сюда можно
    передавать как сброс в бункер B1B6, так и получение материала из R1

    void execute(MachineContext* ctx) override {
        switch (SubState) {
            case 1:
                ctx->controls.MR = true;
                if (!ctx->sensors.DN) {
                    cout << "[!] Аварийная ситуация! Контейнер накренился
во время передвижения!";
                    ctx->changeState(new AlarmState());
                    nextState = nullptr;
                    break;
                }
                if (ctx->sensors.DP[1]) {
                    SubState = 2;
                }
            case 2:
                ctx->controls.MR = false;
                cout << "Прибытие на позицию DP1";
                ctx->changeState(new CheckNormState(nextState));
//успешное выполнение группы S_mplR
                nextState = nullptr;
                break;
            }

        }
        string getName() const override { return "Move_to_R1_Right"; }
    };

class MovingToR2LeftState :public State {
    int SubState = 1;
    State* nextState;
public:
    MovingToR2LeftState(State* next) : nextState(next) {} //сюда можно
    передавать как сброс в бункер B1B6, так и получение материала из R1

    void execute(MachineContext* ctx) override {
        switch (SubState) {
            case 1:
                ctx->controls.ML = true;

```

```

        if (!ctx->sensors.DN) {
            cout << "[!] Аварийная ситуация! Контейнер накренился
во время передвижения!";
            ctx->changeState(new AlarmState());
            nextState = nullptr;
            break;
        }
        if (ctx->sensors.DP[2]) {
            SubState = 2;
        }
    case 2:
        ctx->controls.ML = false;
        cout << "Прибытие на позицию DP2";
        ctx->changeState(new CheckNormState(nextState));
        //успешное выполнение группы S_mp2L
        nextState = nullptr;
        break;
    }

}

string getName() const override { return "Move_to_R2_Left"; }
};

class MovingToR2RightState :public State {
    int SubState = 1;
    State* nextState;
public:
    MovingToR2RightState(State* next) : nextState(next) {}           //сюда
можно передавать как сброс в бункер B1B6, так и получение материала из R1

    void execute(MachineContext* ctx) override {
        switch (SubState) {
            case 1:
                ctx->controls.MR = true;
                if (!ctx->sensors.DN) {
                    cout << "[!] Аварийная ситуация! Контейнер накренился
во время передвижения!";
                    ctx->changeState(new AlarmState());
                    nextState = nullptr;
                    break;
                }
                if (ctx->sensors.DP[2]) {
                    SubState = 2;
                }
            case 2:
                ctx->controls.MR = false;
                cout << "Прибытие на позицию DP2";
                ctx->changeState(new CheckNormState(nextState));
                //успешное выполнение группы S_mp2r
                nextState = nullptr;
                break;
            }

        }

    string getName() const override { return "Move_to_R2_Right"; }
};

class MovingToR3LeftState :public State {
    int SubState = 1;
    State* nextState;
public:
    MovingToR3LeftState(State* next) : nextState(next) {}           //сюда можно
передавать как сброс в бункер B1B6, так и получение материала из R1

```

```

        void execute(MachineContext* ctx) override {
            switch (SubState) {
                case 1:
                    ctx->controls.MR = true;
                    if (!ctx->sensors.DN) {
                        cout << "[!] Аварийная ситуация! Контейнер накренился
во время передвижения!";
                        ctx->changeState(new AlarmState());
                        nextState = nullptr;
                        break;
                    }
                    if (ctx->sensors.DP[3]) {
                        SubState = 2;
                    }
                case 2:
                    ctx->controls.MR = false;
                    cout << "Прибытие на позицию DP3";
                    ctx->changeState(new CheckNormState(nextState)); //успешное
выполнение группы S_мр3r (здесь названа по-другому)
                    nextState = nullptr;
                    break;
            }

            string getName() const override { return "Move_to_R3_Left"; }
};

class Bunker2_Dump :public State {
    int SubState = 1;
    State* nextState;

public:
    Bunker2_Dump(State* next) : nextState(next) {}

    void execute(MachineContext* ctx) override {
        switch (SubState) {
            case 1:
                if (!ctx->sensors.D[2]) { //если контейнер B2 не полон,
то переходим в S_b22
                    SubState = 3;
                }
                else {
                    SubState = 2; //Переходим в
S_Idle_Drain_B2
                }
                break;
            case 2:
                if (!ctx->sensors.D[2]) {
                    SubState = 3; //бункер B2
опустошен, сливаем в бункер
                }
                break;
            case 3:
                ctx->controls.MKL = true; //однозначно задаем
состояния двигателей поворота для надежности
                ctx->controls.MKR = false;
                if (ctx->sensors.DKL) {
                    SubState = 4;
                }
                break;
            case 4:

```

```

        ctx->controls.MKL = false;           //однозначно задаем
состояния двигателей поворота для надежности
        ctx->controls.MKR = false;
        SubState = 5;
        break;
    case 5:
        if (ctx->sensors.DT_OUT) {
            SubState = 6;
        }
        break;
    case 6:
        ctx->controls.MKR = true;           //однозначно задаем
состояния двигателей поворота для надежности
        ctx->controls.MKL = false;
        if (ctx->sensors.DN) {
            SubState = 7;
        }
        break;
    case 7:
        ctx->controls.MKR = false;         //однозначно задаем
состояния двигателей поворота для надежности
        ctx->controls.MKL = false;
        ctx->changeState(nextState);
        nextState = nullptr;
        break;
    }
}
string getName() const override { return "Bunker2_Dump"; }
};

class Bunker5_Dump :public State {
    int SubState = 1;
    State* nextState;

public:
    Bunker5_Dump(State* next) : nextState(next) {}

    void execute(MachineContext* ctx) override {
        switch (SubState) {
            case 1:
                if (!ctx->sensors.D[5]) {    //если контейнер B0 не полон,
то переходим в S_Rub2
                    SubState = 3;
                }
                else {
                    SubState = 2;           //Переходим в
S_Idle_Drain_Rub
                }
                break;
            case 2:
                if (!ctx->sensors.D[5]) {
                    SubState = 3;           //бункер B0
опустошен, сливаем в бункер
                }
                break;
            case 3:
                ctx->controls.MKR = true;   //однозначно задаем
состояния двигателей поворота для надежности
                ctx->controls.MKL = false;
                if (ctx->sensors.DKR) {
                    SubState = 4;
                }

```

```

        break;
    case 4:
        ctx->controls.MKR = false;           //однозначно задаем
состояния двигателей поворота для надежности
        ctx->controls.MKL = false;
        SubState = 5;
        break;
    case 5:
        if (ctx->sensors.DT_OUT) {
            SubState = 6;
        }
        break;
    case 6:
        ctx->controls.MKL = true;           //однозначно задаем
состояния двигателей поворота для надежности
        ctx->controls.MKR = false;
        if (ctx->sensors.DN) {
            SubState = 7;
        }
        break;
    case 7:
        ctx->controls.MKL = false;         //однозначно задаем
состояния двигателей поворота для надежности
        ctx->controls.MKR = false;
        ctx->changeState(nextState);
        nextState = nullptr;
        break;
    }

}

string getName() const override { return "Bunker5_Dump"; }
};

class RubbishDump :public State {
    int SubState = 1;
    State* nextState;

public:
    RubbishDump(State* next) : nextState(next) {}

    void execute(MachineContext* ctx) override {
        switch (SubState) {
            case 1:
                if (!ctx->sensors.D[0]) {    //если контейнер B0 не полон,
то переходим в S_Rub2
                    SubState = 3;
                }
                else {
                    SubState = 2;           //Переходим в
S_Idle_Drain_Rub
                }
                break;
            case 2:
                if (!ctx->sensors.D[0]) {
                    SubState = 3;           //бункер B0
опустошен, сливаем в бункер
                }
                break;
            case 3:
                ctx->controls.MKL = true;   //однозначно задаем
состояния двигателей поворота для надежности

```

```

        ctx->controls.MKR = false;
        if (ctx->sensors.DKL) {
            SubState = 4;
        }
        break;
    case 4:
        ctx->controls.MKL = false;           //однозначно задаем
        ctx->controls.MKR = false;
        SubState = 5;
        break;
    case 5:
        if (ctx->sensors.DT_OUT) {
            SubState = 6;
        }
        break;
    case 6:
        ctx->controls.MKR = true;           //однозначно задаем
        ctx->controls.MKL = false;
        if (ctx->sensors.DN) {
            SubState = 7;
        }
        break;
    case 7:
        ctx->controls.MKR = false;         //однозначно задаем
        ctx->controls.MKL = false;
        ctx->changeState(nextState);
        nextState = nullptr;
        break;
    }

}

string getName() const override { return "Rubbish_Dump"; }
};

class FillingFromR1 :public State {
private:
    int SubState = 1;
    State* nextState; // Владение этим указателем передается от IdleState

public:
    FillingFromR1(State* next) :nextState(next) {}

    //~FillingFromR1() override {}

    void execute(MachineContext* ctx) override {
        switch (SubState) {
            case 1:
                // Включаем клапан
                ctx->controls.V[0] = true;
                if (ctx->sensors.DV[0]) {
                    SubState = 2;
                }
                break;
            case 2:
                ctx->controls.V[0] = true;
                if (ctx->sensors.DT_IN) {
                    ctx->controls.V[0] = false; //сразу выключаем
                }
                SubState = 3;
            }
        }
    }
};

```

для надежности

```

        }
        if (ctx->sensors.DR[0]) {
            ctx->controls.V[0] = false;
            ctx->changeState(new RubbishDump(nextState));
            ctx->controls.V[0] = false;
            nextState = nullptr; // Передаем владение RubbishDump
            break;
        }
        break;
    case 3:
        // Выключаем клапан
        ctx->controls.V[0] = false;

        if (ctx->sensors.AT) {
            if (nextState != nullptr) {
                nextState = nullptr;
            }
            ctx->changeState(new AlarmState());
            break;
        }

        // Ждем закрытия клапана и меняем состояние
        if (!ctx->sensors.DV[0]) {
            ctx->changeState(nextState);
            nextState = nullptr;
        }
        break;
    }
    string getName() const override { return "Filling_From_R1"; }
};

class FillingFromR2 :public State {
    int SubState = 1;
    State* nextState;
public:
    FillingFromR2(State* next) :nextState(next) {}

    void execute(MachineContext* ctx) override {
        switch (SubState) {
            case 1:
                ctx->controls.V[1] = true;
                if (ctx->sensors.DV[1]) {
                    SubState = 2;
                }
                break;
            case 2:
                ctx->controls.V[1] = true;
                if (ctx->sensors.DT_IN) {
                    ctx->controls.V[1] = false;           //сразу выключаем
                }
                SubState = 3;
            }
            if (ctx->sensors.DR[1]) {
                ctx->controls.V[1] = false;
                ctx->changeState(new RubbishDump(nextState));
                //переход в сброс мусора если резервуары пусты
                nextState = nullptr;
                break;
            }
            break;
            case 3:
                ctx->controls.V[1] = false;

```

```

        if (ctx->sensors.AT) {
            if (nextState != nullptr) {
                nextState = nullptr;
            }
            ctx->changeState(new AlarmState());
            break;
        }
        if (!ctx->sensors.DV[1]) {
            ctx->changeState(nextState);
            nextState = nullptr;
        }
        break;
    }
}
string getName() const override { return "Filling_From_R2"; }
};

class FillingFromR3 :public State {
    int SubState = 1;
    State* nextState;
public:
    FillingFromR3(State* next) :nextState(next) {}

    void execute(MachineContext* ctx) override {
        switch (SubState) {
            case 1:
                ctx->controls.V[2] = true;
                if (ctx->sensors.DV[2]) {
                    SubState = 2;
                }
                break;
            case 2:
                ctx->controls.V[2] = true;
                if (ctx->sensors.DT_IN) {
                    ctx->controls.V[2] = false;           //сразу выключаем
кларан R3
                    SubState = 3;
                }
                if (ctx->sensors.DR[2]) {
                    ctx->controls.V[2] = false;
                    ctx->changeState(new RubbishDump(nextState));
                }
                //переход в сброс мусора, если R3 опусутошится
                nextState = nullptr;
                break;
            }
            break;
            case 3:
                ctx->controls.V[2] = false;
                if (ctx->sensors.AT) {
                    if (nextState != nullptr) {
                        nextState = nullptr;
                    }
                    ctx->changeState(new AlarmState());
                }
                if (!ctx->sensors.DV[2]) {
                    ctx->changeState(nextState);
                    nextState = nullptr;
                }
                break;
            }
        }
    }
    string getName() const override { return "Filling_From_R3"; }
};

```

```

class IdleState : public State {
public:
    void execute(MachineContext* ctx) override {
        if (ctx->sensors.DN && ctx->sensors.Start && ctx->sensors.DP[0]) {
            cout << "Запуск цикла..." << endl;

            // СТРОИМ ЦЕПОЧКУ В ОБРАТНОМ ПОРЯДКЕ (как матрешку):
            //
            //12. Возвращаемся в изначальное состояние (DP0)
            State* step12 = new MovingToR0LeftState(new IdleState());

            //11. Сбрасываем в B5
            State* step11 = new Bunker5_Dump(step12);

            //10. Едем к B5
            State* step10 = new MovingToR2LeftState(step11);

            //9. Наполняем контейнер материалом из R3
            State* step9 = new FillingFromR3(step10);

            //8. Едем к R3 за материалом
            State* step8 = new MovingToR3LeftState(step9);

            //7. Не отъезжая, наполняем контейнер материалом из R2
            State* step7 = new FillingFromR2(step8);

            //6. Сбрасываем материал в B2
            State* step6 = new Bunker2_Dump(step7);

            //5. Едем к B2 сбрасывать материал в B2
            State* step5 = new MovingToR2LeftState(step6);

            //4. Наполняем контейнер материалом из R3
            State* step4 = new FillingFromR3(step5);

            // 3. Едем к R3 за материалом
            State* step3 = new MovingToR3LeftState(step4);

            // 2. Перед отъездом нужно наполниться из R1
            State* step2 = new FillingFromR1(step3);

            // 1. Первое действие: Ехать к R1 (вправо). Передаем ему шаг
2.
            State* step1 = new MovingToR1RightState(step2);

            ctx->changeState(step1);
        }
    }
    string getName() const override { return "Idle"; }
};

void AlarmState::execute(MachineContext* ctx) {
    if (ctx->sensors.Reset) {
        cout << "Выход из аварии. Переход в состояние ожидания";
        ctx->changeState(new IdleState());
    }
}

void MachineContext::changeState(State* newState) {
    if (currentState) {
        currentState->exit(this);
    }
}

```

```

        // Используем reset для сырого указателя и передачи владения в
        unique_ptr
        currentState.reset(newState);

        if (currentState) {
            cout << "Переход в состояние: " << currentState->getName() <<
            "\n";
            currentState->enter(this);
        }
    }

void MachineContext::update() {
    if (currentState) {
        if (sensors.AT || sensors.BK0 || sensors.BK1) {
            if (currentState->getName() != "ALARM") {
                cout << "ОБНАРУЖЕНА АВАРИЙНАЯ СИТУАЦИЯ В АСУ!";
                changeState(new AlarmState());
                return;
            }
        }
        currentState->execute(this);
        controls.print();
        cout << endl;
    }
}

void SimulatePhysics(MachineContext& mc, const string& message) {
    cout << "Симуляция:" << message<<"\n";
    this_thread::sleep_for(chrono::milliseconds(1000)); //симулируем
    ожидание для какого-то действия
    mc.update();
}

int main() {
    setlocale(LC_ALL, "Russian");
    MachineContext mc;

    // 1. Инициализация
    mc.changeState(new IdleState());

    // ЭТАП 1: ЗАПУСК
    mc.sensors.Start = true;
    mc.sensors.DP[0] = true;
    SimulatePhysics(mc, "Оператор нажимает кнопку START. Контейнер на базе
    (DP0).");

    // ЭТАП 2: ДВИЖЕНИЕ К R1 (ВПРАВО)
    mc.sensors.DP[0] = false;
    SimulatePhysics(mc, "Контейнер поехал вправо... (DP0=0)");

    mc.sensors.DP[1] = true;
    mc.sensors.DN = true;
    SimulatePhysics(mc, "Прибыли к датчику DP1. Проверка горизонта
    (DN=1).");
    // Переход: MovingToR1Right -> CheckNorm -> FillingFromR1

    // ЭТАП 3: НАПОЛНЕНИЕ ИЗ R1
    // 1. Клапан открывается
    mc.sensors.DV[0] = true; // Сначала ставим датчик
    SimulatePhysics(mc, "Клапан R1 открылся (DV[0]=1).");
    // Update видит DV[0]=true -> переходит к ожиданию наполнения

    // 2. Бак наполняется

```

```

mc.sensors.DT_IN = true; // Сначала ставим датчик уровня
SimulatePhysics(mc, "Бак наполнился! (DT_IN=1).");
// Update видит DT_IN=1 -> дает команду закрыть клапан (переход в Case
3)

// 3. Клапан закрывается
mc.sensors.DV[0] = false; // Физическое закрытие
mc.sensors.DT_IN = false;
SimulatePhysics(mc, "Клапан R1 закрылся (DV[0]=0).");
// Update видит Case 3 -> Выключает V[0] -> Проверяет DV[0]=0 ->
Переходит дальше
//
// ЭТАП 4: ДВИЖЕНИЕ К R3 (ВПРАВО)
mc.sensors.DP[1] = false;
SimulatePhysics(mc, "Контейнер уехал с DP1 к R3...");

mc.sensors.DP[3] = true;
mc.sensors.DN = true;
SimulatePhysics(mc, "Прибыли к датчику DP3. Проверка DN.");

// ЭТАП 5: НАПОЛНЕНИЕ ИЗ R3
mc.sensors.DV[2] = true;
SimulatePhysics(mc, "Клапан R3 открылся (DV[2]=1).");

mc.sensors.DT_IN = true;
SimulatePhysics(mc, "Бак наполнился (DT_IN=1).");

mc.sensors.DV[2] = false;
mc.sensors.DT_IN = false;
SimulatePhysics(mc, "Клапан R3 закрылся (DV[2]=0).");

// ЭТАП 6: ДВИЖЕНИЕ К R2 (ВЛЕВО)
mc.sensors.DP[3] = false;
SimulatePhysics(mc, "Контейнер едет влево к R2...");

mc.sensors.DP[2] = true;
mc.sensors.DN = true;
SimulatePhysics(mc, "Прибыли к датчику DP2. Проверка DN.");

// ЭТАП 7: СБРОС В БУНКЕР 2
mc.sensors.D[2] = false;
SimulatePhysics(mc, "Проверка бункера: он пуст (D[2]=0). Начинаем
сброс.");

mc.sensors.DKL = true;
mc.sensors.DN = false;
SimulatePhysics(mc, "Контейнер наклонился влево (DKL=1). DN=0.");
SimulatePhysics(mc, "Выключение мотора наклона");

mc.sensors.DT_OUT = true;
cout << "[Визуализация] Материал высыпается..." << endl;
// Тут можно сделать доп. update, пока сыпется
SimulatePhysics(mc, "Датчик пролета материала сработал (DT_OUT=1)");

// Возврат в исходное
mc.sensors.DT_OUT = false; // Мусор высыпался
SimulatePhysics(mc, "Начало возврата в горизонт (DN=0)");
mc.sensors.DKL = false; // Больше не наклонен

mc.sensors.DN = true; // Вернулся в горизонт
SimulatePhysics(mc, "Контейнер вернулся в горизонт (DN=1).");

```

```

// ЭТАП 8: НАПОЛНЕНИЕ ИЗ R2
mc.sensors.DV[1] = true;
SimulatePhysics(mc, "Клапан R2 открылся (DV[1]=1).");

mc.sensors.DT_IN = true;
SimulatePhysics(mc, "Бак наполнился (DT_IN=1).");

mc.sensors.DV[1] = false;
SimulatePhysics(mc, "Клапан R2 закрыт, ожидаем фи");
mc.sensors.DT_IN = false;
SimulatePhysics(mc, "Клапан R2 закрылся (DV[1]=0).");

// ЭТАП 9: ДВИЖЕНИЕ К R3 (СНОВА)
mc.sensors.DP[2] = false;
SimulatePhysics(mc, "Контейнер едет к R3...");

mc.sensors.DP[3] = true;
mc.sensors.DN = true;
SimulatePhysics(mc, "Прибыли на DP3. Проверка DN.");

// ЭТАП 10: НАПОЛНЕНИЕ ИЗ R3 (ПОВТОРНО)
mc.sensors.DV[2] = true;
SimulatePhysics(mc, "Клапан R3 открылся (DV[2]=1).");

mc.sensors.DT_IN = true;
SimulatePhysics(mc, "Бак наполнился (DT_IN=1).");

mc.sensors.DV[2] = false;
mc.sensors.DT_IN = false;
SimulatePhysics(mc, "Клапан R3 закрылся (DV[2]=0).");

// ЭТАП 11: ВОЗВРАТ К R2 И СБРОС В B5
mc.sensors.DP[3] = false;
SimulatePhysics(mc, "Едем обратно к R2...");

mc.sensors.DP[2] = true;
mc.sensors.DN = true;
SimulatePhysics(mc, "Прибыли на DP2. Переход к сбросу в B5.");

// Логика сброса в Bunker 5
mc.sensors.D[5] = false; // Бункер 5 пуст
SimulatePhysics(mc, "Проверка B5. Пуст. Начинаем сброс.");

mc.sensors.DKR = true; // Наклон ВПРАВО
mc.sensors.DN = false;
SimulatePhysics(mc, "Контейнер наклонился вправо (DKR=1).");
SimulatePhysics(mc, "Выключение мотора наклона");

mc.sensors.DT_OUT = true;
SimulatePhysics(mc, "Материал высыпался (DT_OUT=1).");

// Возврат
mc.sensors.DT_OUT = false;
SimulatePhysics(mc, "Начало возврата в горизонт");
mc.sensors.DKR = false;
mc.sensors.DN = true;
SimulatePhysics(mc, "Контейнер вернулся в горизонт (DN=1).");

// ЭТАП 12: ВОЗВРАТ НА БАЗУ R0
mc.sensors.DP[2] = false;
SimulatePhysics(mc, "Финишная прямая: едем на базу R0...");

mc.sensors.DP[0] = true;

```

```
mc.sensors.DN = true;  
SimulatePhysics(mc, "Прибыли на базу DP0. Цикл завершен.");  
  
cout << "ТЕХНОЛОГИЧЕСКИЙ ЦИКЛ ЗАВЕРШЕН!" << endl;  
mc.sensors.Start = false;  
return 0;  
}
```